

GraphOne / FrontierAtlas — Intelligence Graph Ingestion Pipeline

Architecture & Production Design Document — Phase VI

1. Scale Strategy — 500,000+ Startups, Products, and Papers

The core principle: **volume is an infrastructure knob, not a code change**. Every crawl unit (one URL, one API page) is an independent, idempotent task pushed onto a durable queue. A fixed pool of stateless async workers pulls from that queue, so going from 1,000 to 500,000 records means adding workers and queue partitions, not touching pipeline logic — the same `Pipeline.run_url_batch()` path handles both.

Source-tier acquisition strategy:

- **Official bulk APIs first.** arXiv's OAI-PMH/Atom export and the Papers-with-Code REST API return thousands of records per request with stable pagination — this is how the 1,000+ research-paper minimum is actually hit, not by rendering HTML pages one at a time. HTML scraping is reserved for sources with no bulk channel.
- **Directory pagination via the shared worker pool** for startup/product directories (e.g. YC company lists, Product Hunt categories): enumerate the paginated index once, fan the resulting detail-page URLs out across the queue.
- **Horizontal scaling path:** single-machine run uses an in-process `asyncio.Queue` (current repo state, good to ~10K–50K records). Beyond that, the same queue contract moves to Redis (or SQS) so multiple worker *processes*, then multiple worker *machines*, can pull from it — no pipeline code changes, only the queue backend and a config value (`NUM_WORKERS`, `GLOBAL_CONCURRENCY`).
- **No manual intervention:** a scheduler (cron / Airflow DAG) re-enqueues incremental pagination cursors per source; failures requeue with backoff rather than halting the run (see error handling below).

2. Handling 413 Payload Too Large & 429 Rate Limits

429 — rate limits, across thousands of concurrent extractions:

- Full-jitter exponential backoff (`utils/backoff.py`): `delay = random(0, min(max_seconds, base * 2^attempt))`. Jitter, not plain exponential backoff, matters specifically at high concurrency — a fixed schedule causes thousands of coroutines to retry in synchronized waves that immediately re-trip the limit.
- Provider Retry-After headers are authoritative when present and override the computed delay.
- On repeated 429s past `max_retries`, the LLM orchestrator falls back to the next tier in the chain (Gemini Flash → Groq Llama 3 → DeepSeek) rather than blocking the worker — a single saturated provider degrades throughput, it doesn't stall the pipeline.
- Per-host semaphores in the crawler (not just a global cap) mean one aggressively-limiting site throttles only its own worker slice.

413 — context window overflows:

- Pre-emptive chunking (`llm/chunking.py`): boilerplate (nav/cookie banners/repeated whitespace) is stripped first, then text is split on paragraph boundaries to stay under a conservative token budget (default 6,000 est. tokens) — well under every target provider's actual limit, so a 413 should be rare in steady state.
- 150-token trailing overlap between chunks so a fact split across a boundary (a claim and its number in adjacent paragraphs) isn't lost to either chunk.
- If a 413 fires anyway (a provider's real limit is stricter than assumed), the orchestrator re-chunks just that piece far smaller (1,500 tokens) and retries the same provider before falling back to the next tier — reactive correction, not just prevention.

- Per-chunk extractions are merged with a *first-non-null-wins* strategy per field, with list fields (authors, entities) unioned rather than overwritten.

3. Freshness Tracking Across Distributed Crawler Nodes

Two layers work together so the same article/job is never double-processed, even with many crawler nodes running concurrently:

- **Explicit date normalization** (utils/date_utils.py): structured metadata (JSON-LD, <meta> tags) is preferred; visible relative text ("2 hours ago") is parsed against the crawl's reference_time, not wall-clock now(), so results are deterministic and reproducible across nodes. A record is only placed in the strict 24-hour-fresh tabs when is_fresh_24h is unambiguously true — never guessed.
- **Shared dedup ledger, not per-node memory:** the {url: content_hash} map (seen_hashes in the current single-node code) is backed by Redis (SETNX url content_hash) in the distributed deployment — an atomic check-and-set means two nodes racing on the same URL still only produce one record. This is also the heuristic fallback for sources with no reliable date signal at all: if the content hash for a URL is unchanged since the last run, it's treated as already-seen and dropped, no timestamp guessing involved.
- **Idempotent writes downstream:** the Sheets/DB write layer keys on source.url + schemaVersion, so even a duplicate that slips past the ledger (e.g. a hash-collision edge case) becomes an upsert, not a duplicate row.

4. Storage Strategy

Primary store: PostgreSQL. The canonical entity records (startups, products, papers, jobs, news) are relational at heart — strong schema guarantees matter here precisely because hallucination-free, source-traceable data is the whole point of this system, and Postgres's constraints (NOT NULL on source.url, foreign keys from Product → Startup) enforce that at the database layer, not just in application code. JSONB columns hold the less-structured content.data payloads for schema evolution without migrations on every new field.

Graph layer: Neo4j (or Postgres + Apache AGE if minimizing infra surface). The core product value here is *relationships* — Startup→Product, Paper→GitHub repo, Person→Startup, Company→Job — and graph queries ("find all papers connected to startups founded by ex-OpenAI researchers") are what a relational join chain gets slow and awkward at as the graph deepens. Postgres remains the source of truth; the graph store is a queryable projection kept in sync via the entity-resolution write path.

Vector layer: pgvector (co-located with Postgres) rather than a separate vector DB. At this stage semantic search (paper similarity, product/competitor clustering) doesn't need a dedicated vector database's extra operational surface — pgvector keeps embeddings next to the records they describe and one fewer system to keep consistent. Revisit a dedicated store (Qdrant/Pinecone) only if embedding volume or query latency actually outgrows it.

Object storage: S3-compatible bucket for raw HTML/PDF snapshots at crawl time — re-running LLM extraction after a prompt improvement shouldn't require re-crawling the web.

See README.md for setup instructions and src/ for the corresponding implementation of each section above.